

Section 6: Hands on Part 2 Useful Tools

Robert Underwood, Argonne National Laboratory and the University of Chicago

How do we use compressors?

```
confparams_cpr->errorBoundMode = ABS;
confparams_cpr->absErrBound = 1e-4;
unsigned char *bytes = SZ_compress(SZ_FLOAT, (float *)data, &outsize, dim[4],
                                   dim[3], dim[2], dim[1], dim[0]);
```

```
zfp_stream_set_accuracy(zfp, 1e-4);
size_t outsize = zfp_compress(zfp, in_field);
```

```
unsigned char *bytes = mgard_compress(data, outsize, mgard_dims[0],
                                      mgard_dims[1], mgard_dims[2], 1e-4f);
```

Compressors have too many interfaces!

Is this thread-safe???

Modifies input sometimes

```
confparams_cpr->errorBoundMode = ABS;
confparams_cpr->absErrBound = 1e-4;
unsigned char *bytes = SZ_compress(SZ_FLOAT, (float *)data, &outsize, dim[4],
                                   dim[3], dim[2], dim[1], dim[0]);
```

Global configuration struct

Pointwise absolute error bound named differently

Dimensions provided in opposite order

```
zfp_stream_set_accuracy(zfp, 1e-4);
size_t outsize = zfp_compress(zfp, in_field);
```

Compressed Stream returned "implicity"

Local configuration

Uses 1's for unused dimensions

```
unsigned char *bytes = mgard_compress(data, outsize, mgard_dims[0],
                                      mgard_dims[1], mgard_dims[2], 1e-4f);
```

Configuration in function arguments

Compressors have too many interfaces!

Is this thread-safe???

Modifies input sometimes

```
confparams_cpr->errorBoundMode = ABS;
confparams_cpr->absErrBound = 1e-4;
unsigned char *bytes = SZ_compress(SZ_FLOAT, (float *)data, &outsize, dim[4],
                                   dim[3], dim[2], dim[1], dim[0]);
```

Global configuration struct

Pointwise absolute error bound named differently

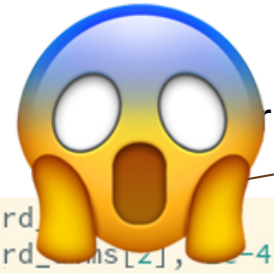
Dimensions provided in opposite order

Only 3 Compressors!

```
zfp_stream_set_accuracy(zfp, 1e-4)
size_t outsize = zfp_compress(zfp, in_field);
```

Compressed Stream returned "implicitly"

Local configuration



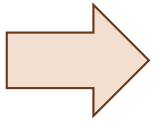
unused dimensions

```
unsigned char *bytes = mgard_compress(data, outsize, mgard
                                      mgard_dims[1], mgard_dims[2], 1e-4f);
```

Configuration in function arguments

Compressors have too many interfaces!

- Gaps:
  - There isn't a unified, stable interface for compression
  - The interfaces are hard to configure and optimize to achieve portable performance



- Challenge:
  - There are many incompatible interfaces
    - dimension ordering
    - is the input modified?
  - This is the same kind of challenge that led to the MPI standard in message passing

Where does HPC use compressors?

1. Exascale Applications
   - AutoSFX, HACC, NYX, GAMESS, ExaFEL, QMC-PACK, XGC, etc...
2. IO libraries Plugins
   - HDF5, ADIOS2
3. Command Line Interfaces
   - SZ, ZFP, MGARD, etc...
4. Bound User Metrics
   - FRaZ, MGARD-QOI
5. Distributed Experiments
   - One-off scripts, Cheetah/Savanna
6. Quality Analysis Tools
   - Z-Checker, Foresight
7. Test Coverage and Fuzzing
   - presiso_fuzz
8. Language Bindings
   - C/C++, Julia, R, Rust, Python
9. And more!!

LibPressio and Productivity

Can we construct an efficient abstraction for lossy compression of scientific data?

Our Contributions

We present LibPressio a productive and performance generic compression library. Our work includes:

1. An extensive comparison of 9 existing interfaces to show performance impacts of design decisions
2. Measurement of the overhead of design decisions in modern libraries, while demonstrating *de minimis* overhead
3. A 50 - 90% reduction in the volume of client code while maintaining at least feature parity, and often more features

LibPressio Provides a Common Interface

- Common Abstractions for:
  - Loading compressors
  - Configuration
  - Compression/Decompression
  - Representing Data
  - Error Reporting
  - Computing Metrics

Get LibPressio

LibPressio Provides a Common Interface

- Common Abstractions for:
  - Loading compressors
  - Configuration
  - Compression/Decompression
  - Representing Data
  - Error Reporting
  - Computing Metrics

Get LibPressio

```
 // get the compressor
 struct pressio *library = pressio_instance();
 struct pressio_compressor *comp =
pressio_get_compressor(library, "pressio");

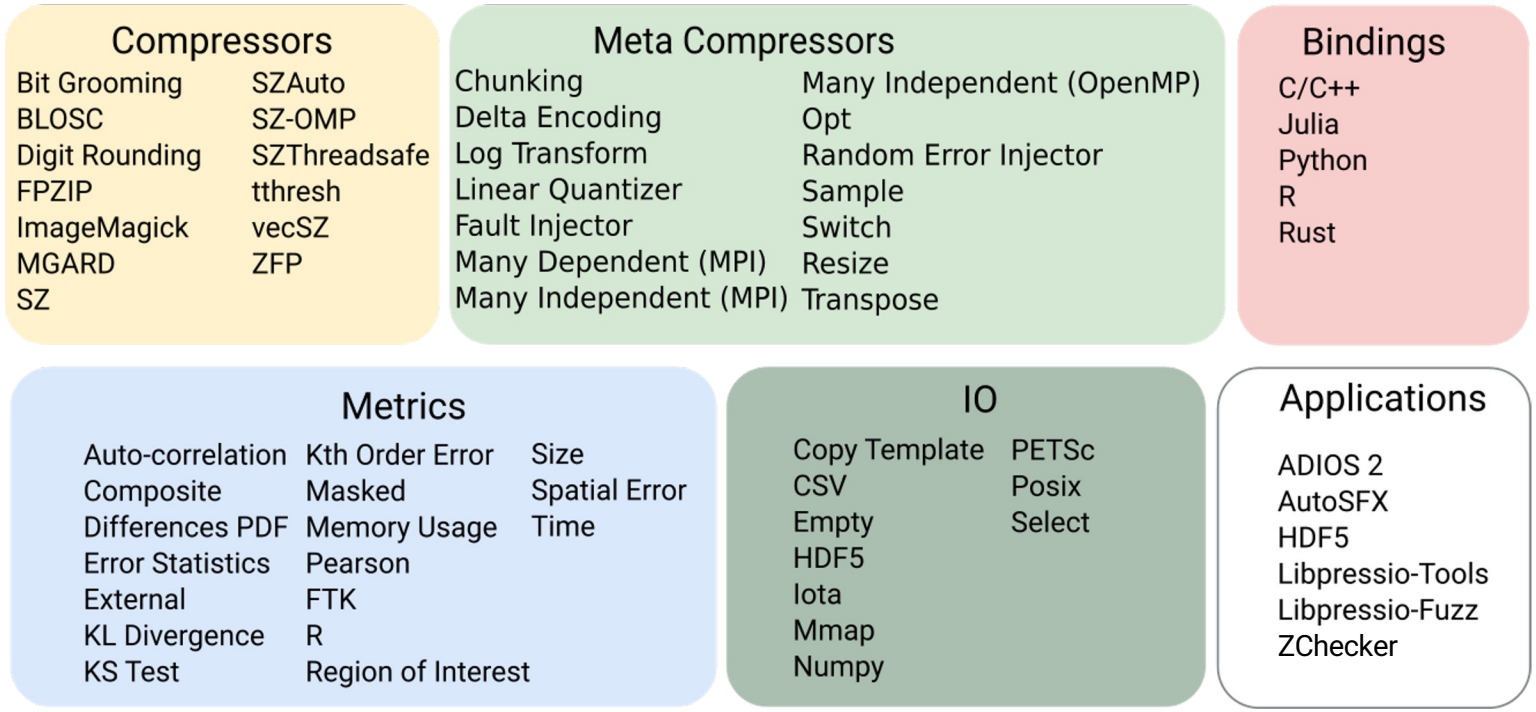
 // configure and validate the options
 struct pressio_options *options = pressio_options_new();
 pressio_options_set_string(options, "pressio:compressor", "sz");
 pressio_options_set_double(options, "pressio:abs", 1e-3);
 pressio_compressor_set_options(comp, options);

 // read in an input buffer
 size_t dims[] = {500, 500, 100};
 struct pressio_data *metadata =
     pressio_data_new_empty(pressio_float_dtype, 3, dims);
 struct pressio_data *input_data =
     pressio_io_data_path_read(metadata, "CLOUDf48.bin.f32");

 // create output buffers
 struct pressio_data *comp_data =
     pressio_data_new_empty(pressio_byte_dtype, 0, nullptr);
 struct pressio_data *output =
     pressio_data_new_owning(pressio_byte_dtype, 3, dims);

 // compress and decompress data
 pressio_compressor_compress(comp, input_data, comp_data);
 pressio_compressor_decompress(comp, comp_data, output);
```

Current Plugins and Tools



Code that you don't have to write!

Prior Attempts at Unified Interfaces

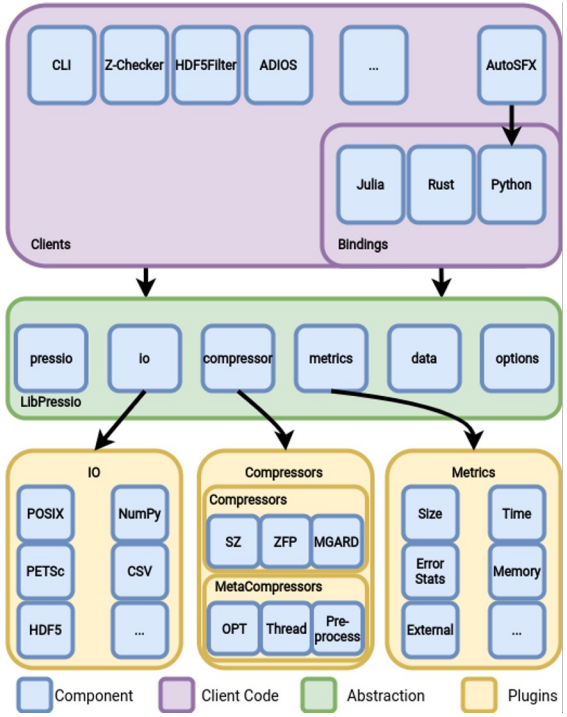
Approach	Dimension aware	Datatype aware	Embeddable	Arbitrary Configuration for Compressors (i.e. cudaStream_t, MPI_Comm)	Introspection	3rd party plugins
Foresight/ CBench	✅	❌	🚧	❌	❌	❌
FFMPEG	✅	🚧	✅	✅	❌	❌
HDF5	✅	✅	✅	❌	✅	✅
ImageMagick	✅	✅	✅	❌	✅	✅
NumCodecs	✅	✅	❌	❌	✅	✅
libarchive	❌	❌	✅	❌	❌	❌
Z-checker	🚧	✅	🚧	❌	❌	✅
SCIL	✅	✅	✅	❌	✅	❌
LibPressio	✅	✅	✅	✅	✅	✅

Prior Attempts at Unified Interfaces

Approach	Dimension aware	Datatype aware	Embeddable	Arbitrary Configuration for Compressors (i.e. cudaStream_t, MPI_Comm)	Introspection	3rd party plugins
Foresight/ CBench	✅	❌	🚧	❌	❌	❌
FFMPEG	✅	🚧	✅	✅	❌	❌
HDF5	✅	✅	✅	❌	✅	✅
ImageMagick	✅	✅	✅	❌	✅	✅
NumCodecs	✅	✅	❌	❌	✅	✅
libarchive	❌	❌	✅	❌	❌	❌
Z-checker	🚧	✅	🚧	❌	❌	✅
SCIL	✅	✅	✅	❌	✅	❌
LibPressio	✅	✅	✅	✅	✅	✅
😱	~1.4x - 1.8x worse CR	~ 1.8x worse CR	~ 1.2x - 2x slower	~ 1.9x slower	≥1.5x More and less complete client code	Duplicates effort

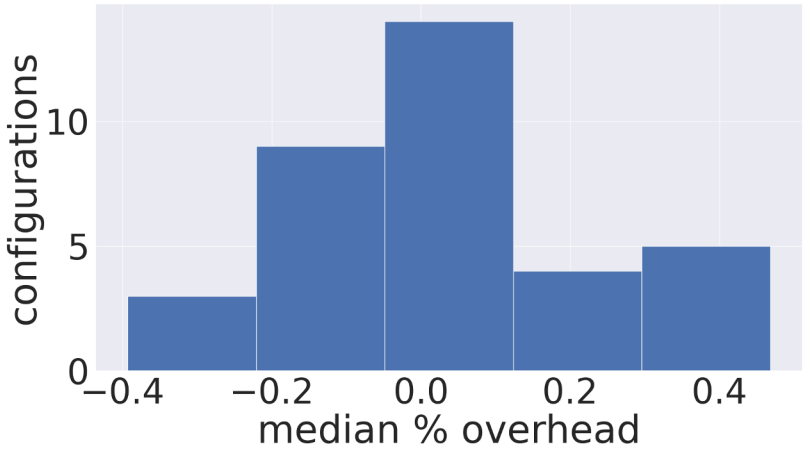
The Design of LibPressio

- 6 Major Components
  - Plugin manager (pressio)
  - Options Abstraction
  - Data Abstraction
  - Compressors Plugins
  - Metrics Plugins
  - IO plugins Plugins



LibPressio Has Little to No Overhead

- Tested 35 configurations with 30 replicates
  - Largest Overhead 2.08%
  - Largest Median 0.47%
  - Largest Variation -1.6% to 1.7%
- Insufficient Evidence to conclude the runtimes meaningfully differ (p=.6, wilcoxon sign rank)



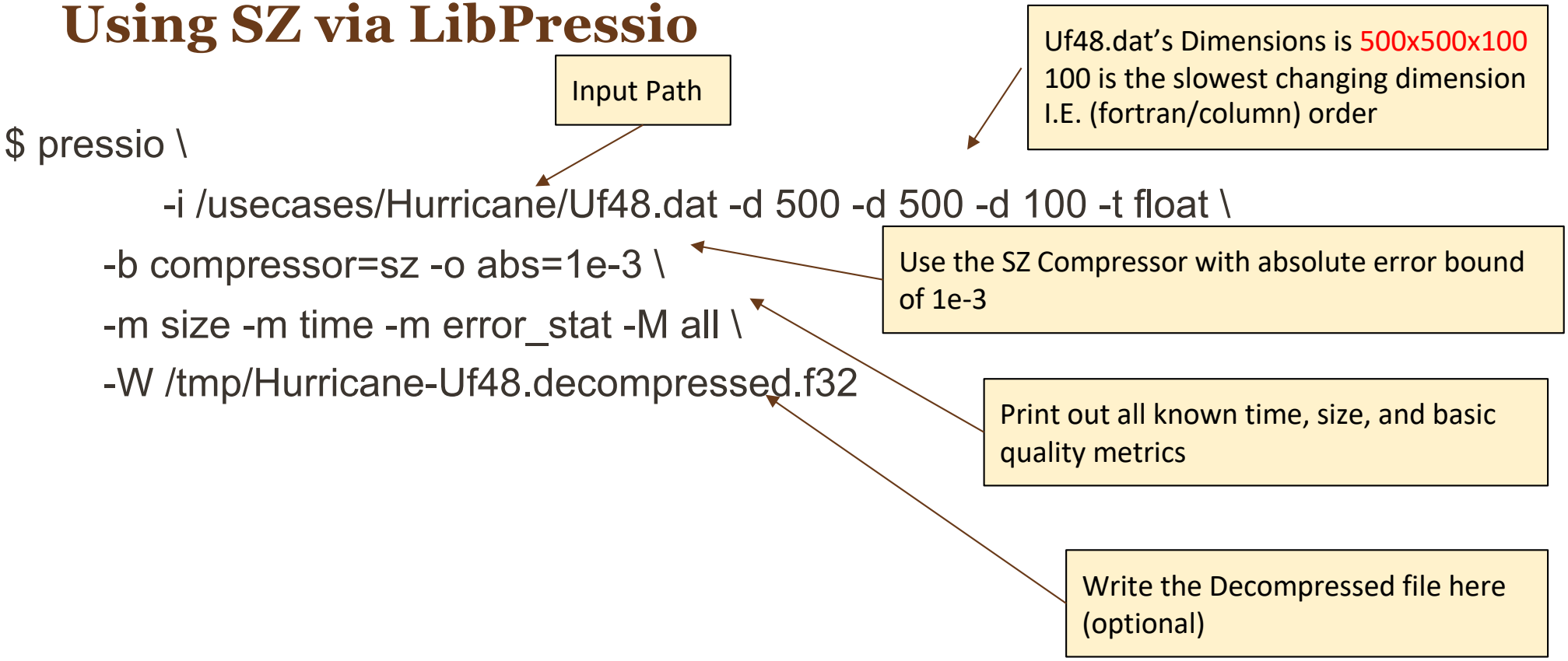
LibPressio and Developer Productivity

- Writing for multiple compressors is hard
- Switching to libpressio
  - Provides a consistent interface
  - Reduced lines of client code by 67% on average
  - Added support for up to 26 compressors and meta-compressors

Task	Compressors	Lines Native	Lines LibPressio	Improvement	Relative Improvement
ADIOS2	3	744	367	377	50.67%
BindingJulia	1	299	25	274	91.64%
BindingPython †	2	768	363	405	52.73%
BindingR	-	-	793	-	-
BindingRust	1	112	34	78	69.64%
CLI †	3	1649	756	893	54.15%
Configuration Optimizer	1	4683	1869	2814	60.09%
DistributedExperiment	-	-	613	-	-
Fuzzer	-	-	24	-	-
HDF5 filter †	2	1469	438	1031	70.18%
Z-Checker	7	3052	405	2647	86.73%

A † indicates that no multi-compressor native solution exists

Using SZ via LibPressio



Using SZ via LibPressio (Sample Output)

$ pressio -i /usecases/Hurricane/Uf48.dat -d 500 -d 500 -d 100 -t float -b compressor=sz -o abs=1e-3 -m size -m time -m error_stat -M all

```
error_stat:average_difference <double> = -8.10589e-08
error_stat:average_error <double> = 0.000497312
error_stat:difference_range <double> = 0.00199999
error_stat:error_range <double> = 0.000999998
error_stat:max_error <double> = 0.000999998
error_stat:max_pw_rel_error <double> = 375.627
error_stat:max_rel_error <double> = 1.08014e-05
error_stat:min_error <double> = 0
error_stat:min_pw_rel_error <double> = 0
error_stat:min_rel_error <double> = 0
error_stat:mse <double> = 3.31488e-07
error_stat:n <uint64> = 25000000
error_stat:psnr <double> = 104.126
error_stat:rmse <double> = 0.00057575
<snip>
size:bit_rate <double> = 6.12524
size:compressed_size <uint64> = 19141366
size:compression_ratio <double> = 5.22429
size:decompressed_size <uint64> = 100000000
```

Basic quality statistics

You can even get compressor specific metrics where provided!

Size information in bytes

Compression time

What if the dimensions are passed in the wrong order?

$ pressio -i /usecases/Hurricane/Uf48.dat -d 500 -d 500 -d 100 -t float -b compressor=sz -o abs=1e-3 -m size -M "size:compression_ratio"

```
size:compression_ratio<double> = 5.22429
```

Passed dimensions incorrectly in C/Row Major Order

$ pressio -i /usecases/Hurricane/Uf48.dat -d 100 -d 500 -d 500 -t float -b compressor=sz -o abs=1e-3 -m size -M "size:compression_ratio"

```
size:compression_ratio<double> = 3.14829
```

Compression Ratio is ≈1.66x worse
Could even be worse depending on dataset and configuration

Using Lossless ZFP via Lib...o

```
$ pressio \
        -i /usecases/Hurricane/Uf48.dat -d 500 -d 500 -d 1
    -b compressor=zfp -o lossless=1 \
    -m size -m time -m error_stat-M all \
    -W /tmp/Hurricane-Uf48.decompressed.f32
```

Input Path

Uf48.dat's Dimensions is 500x500x100
100 is the slowest changing dimension
I.E. (fortran/column) order

Use the ZFP's reversible mode, using the generic name

Print out some time and size metrics

Write the Decompressed file here

Testing ZFP's other modes

Fixed Accuracy Mode (absolute error bound)

$ pressio … -b compressor=zfp -o abs=0.1

$ pressio … -b compressor=zfp -o zfp:accuracy=0.1

Fixed Precision Mode (number of bits of precision)

$ pressio … -b compressor=zfp -o zfp:precision=16

Fixed Rate Mode (number of compressed bits/value)

$ pressio … -b compressor=zfp -o zfp:rate=5

Both of these work. The first works with any compressor that supports an absolute error bound

Getting help on a compressor

$ pressio -a help -b compressor=zfp

```
ZFP is an error bounded lossy compressor that uses a transform which is similar to a discrete cosine transform. More information on
ZFP can be found on its  [project homepage](https://zfp.readthedocs.io/en/release0.5.5/)

Options
<snip>
pressio:abs <double> a pointwise absolute error bound

 compressors may provide this value without supporting abs=0.
 compressors that support abs=0, additionally should also define pressio:lossless

pressio:lossless <int32> use lossless compression,
   the smaller the number the more biased towards speed,
   the larger the number the more biased towards compression

   at this time (may change in the future), individual lossless compressors may internet values less than 1 or greater than 9
differently

pressio:metric <char*> metrics to collect when using the compressor
zfp:accuracy <double> absolute error tolerance for fixed-accuracy mode
zfp:dims <uint32> the dimensionality of the input data, used in fixed-rate mode
zfp:execution <int32> which execution mode to use
zfp:execution_name <char*> which execution mode to use as a human readable string {<char*[]> = {omp, cuda, serial, }}
zfp:maxbits <uint32> maximum number of bits to store per block
zfp:maxprec <uint32> maximum number of bit planes to store
zfp:metric <char*> metrics to collect when using the compressor
zfp:minbits <uint32> minimum number of bits to store per block
zfp:minexp <int32> minimum floating point bit plane number to store
zfp:mode <uint32> a compact encoding of compressor parameters
zfp:omp_chunk_size <uint32> OpenMP chunk size used in OpenMP mode
zfp:omp_threads <uint32> number of OpenMP threads to use in OpenMP mode
zfp:precision <uint32> The precision specifies how many uncompressed bits per value to store, and indirectly governs
the relative error
zfp:rate <double> the rate used in fixed rate mode
zfp:reversible <uint32> use reversible mode
zfp:type <uint32> the type used in fixed rate mode
zfp:wra <int32> write random access used in fixed rate mode
```

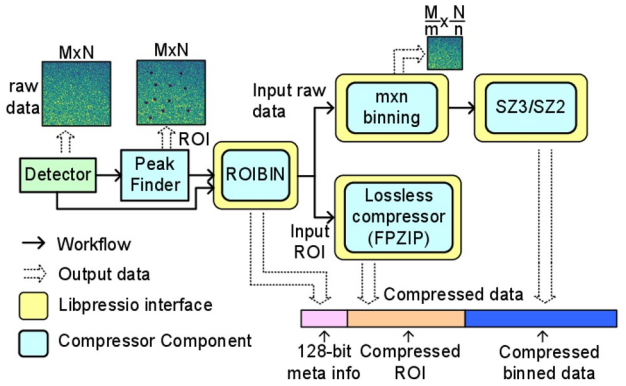
Meta Compressors Boost Productivity

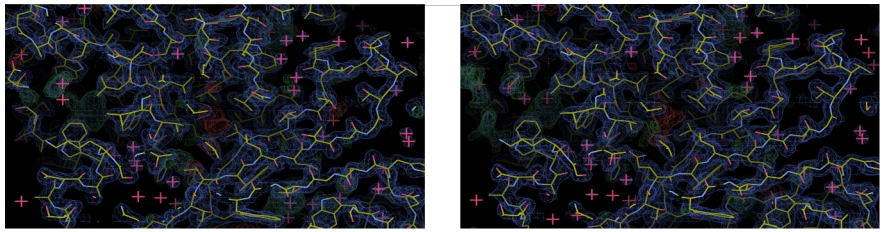
- Not a compressor themselves
- Provide common services:
  - Scripting interface
  - Auto Configuration tools
  - Pre/Post Processors
  - Parallel Runtimes

Case Study: Specialized Compression For ExaFEL

For example, ROIBIN-SZ3 composes binning + SZ compression with lossless preservation of key regions of interest to achieve

- between 46 and 196x compression ratio
- a 17x improvement over the leading lossy compressor
- and 329x over lossless (state of practice)
- without loss in scientific fidelity.

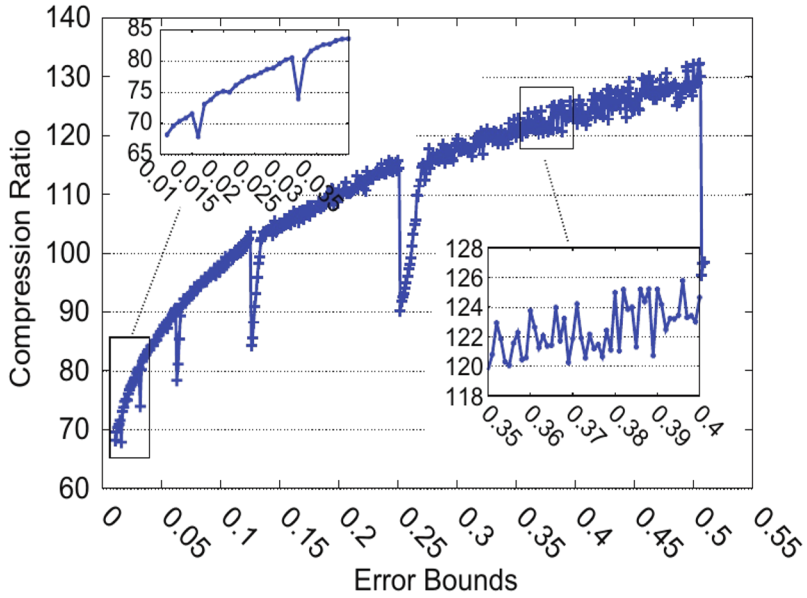




Reconstructed Electron Densities for SE-SAD (left) Original (right) ROBIN-SZ3

Case Study: Configuration Search

- Why not use binary search?
  - It doesn't work consistently
  - The relationship between error bounds and metrics is often not monotonic
- Why not use analytic based methods/specialized compressors
  - Expensive to develop
  - Can we get sufficient results cheaper?



Case Study: Provenance

LibPressio comes with a built-in provence system called *manifest* that

- tracks what compression settings were used
- each time the data was compressed


```
{
  "c": {
    "/p": "0:0",
    "/p/pressio": "0:0",
    "/p/pressio/sz": "0:2"
  },
  "d": [
    500,
    500,
    100
  ],
  "n": "",
  "o": {
    "manifest:compressor": "pressio",
    "manifest:lineage": "",
    "manifest:metric": "composite",
    "manifest:record_lineage_on_decompress": false,
    "pressio:abs": 0.0001,
    "pressio:compressor": "sz",
    "pressio:lossless": {
      "type": 1,
      "value": null
    },
    "pressio:metric": "noop",
    "sz:abs_err_bound": 0.0001,
    "sz:app": "SZ",
    // trimmed for space
  "t": 1
}
```

LibPressio: Results

- Can we construct an efficient abstraction for lossy compression of scientific data?
  - Yes, we can save 50%+ of the lines of client code and reduce switching costs
  - We can have statistically insignificant overhead